

* Assignment:- 2 *

i] Describe the structure of 'C' Program:

→ C program's structure can be broken down into several key sections:-

i] Header section:

- Includes header files using the #include preprocessor directive.
- It contains declarations of functions, variables, & macros.
- Eg: #include <stdio.h>

ii] Main Function:

- Entry point of program. Execution begins here.
- Defined using int main() (returns an integer) or void main() (no return value).
- Contains core logic of program.

iii] Variable Declaration:

- Declares variables within functions (local variables) or outside (global variables).
- Specifies data types & name of variable.
- Eg: int age; float salary;

iv] Body of function:

- Contains executable statements of program.
- Statements perform operations, calculations, & control the flow of execution.
- Enclosed within curly braces {}.

v] Return Statement:

- Used to return value from function to the calling function.
- In the main() function, return 0; typically indicates successful execution.
- Eg: return 0;

C HEADER FILES & THEIR PURPOSES

Header File	Purpose
stdio.h	Provides functions for standard input and output operations (e.g., printf(), scanf()).
stdlib.h	Provides general utility functions (e.g., memory allocation, process control, conversions).
string.h	Provides functions for manipulating strings (e.g., strcpy(), strlen(), strcmp()).
math.h	Provides mathematical functions (e.g., sqrt(), pow(), sin(), cos()).
conio.h	Provides functions for console-related operations (e.g., clrscr(), getch(), gotoxy()) - <i>Non-standard, compiler-specific.</i>
ctype.h	Provides functions to classify and transform characters (e.g., isalpha(), isdigit(), toupper()).
time.h	Provides functions for working with date and time (e.g., time(), localtime()).
assert.h	Provides the assert() macro for debugging.

errno.h	Provides macros for reporting error conditions.
float.h	Defines limits for floating-point types.
limits.h	Defines limits for integer types.
locale.h	Provides functions for cultural and language-specific settings.
setjmp.h	Provides functions for non-local jumps (e.g., setjmp(), longjmp()).
signal.h	Provides functions for handling signals (e.g., signal(), raise()).
stdarg.h	Provides functions for handling variable-length argument lists.
stddef.h	Defines several commonly used data types (e.g., size_t, ptrdiff_t).
iso646.h	Defines macros for alternative spellings of operators and keywords (e.g., and for &&, or for `
wchar.h	Provides functions for working with wide characters.
wctype.h	Provides functions for classifying wide characters.

1] Hello World Program :-

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    printf("Hello World");
```

```
    return 0;
```

```
}
```

2] Sum of Two Numbers Program :-

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int a, b, sum;
```

```
    printf("Enter two numbers: ");
```

```
    scanf("%d %d", &a, &b);
```

```
    sum = a + b;
```

```
    printf("sum : %d\n", sum);
```

```
    return 0;
```

```
}
```

3] Greater Number Program:-

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int a, b;
```

```
    printf("Enter two numbers: ");
```

```
    scanf("%d %d", &a, &b);
```

```
    if (a > b)
```

```
        printf("Greater no. is: %d\n", a);
```

```
    else if (b > a)
```

```
        printf("Greater no. is: %d\n", b);
```

```
    else
```

```
        printf("Both numbers are equal.");
```

```
    return 0;
```

```
}
```


2] Input and Output Functions in C :-

In C, input ~~output~~ functions are used to take user input, & output functions are used to display output.

These are defined in the `stdio.h` library.

i] Input functions:

a] `scanf()`: used to take input from user.

syntax: `scanf("format-specifier", &variable);`

b] `gets()`: used to take full string input.

syntax: `(char str[50]);`

`gets(str);`

c] `fgets()`: Take string input safely, prevents buffer overflow.

syntax: `fgets(variable, size, stdin);`

ii] Output functions:

a] `printf()`: used to display output.

syntax: `printf("format-specifier", variable);`

b] `puts()`: used to print string with new line.

syntax: `puts(variable);`

4] Student information :-

```
#include <stdio.h>
int main ()
{
    int roll_no, num_subjects;
    float marks[10], sum=0, avg;

    printf ("Enter Roll Number: ");
    scanf ("%d", &roll_no);
    printf ("Enter number of subjects: ");
    scanf ("%d", &num_subjects);
    printf ("Enter marks for %d subjects: \n", num_subjects);
```

```
#include <stdio.h>
```

```
int main ()
```

```
{
```

```
    int roll_no;
```

```
    float mark1, mark2, mark3, avg;
```

```
    printf ("Enter roll no. : ");
```

```
    scanf ("%d", &roll_no);
```

```
    printf ("Enter mark of subject 1:");
```

```
    scanf ("%f", &mark1);
```

```
    printf ("Enter mark of subject 2: ");
```

```
    scanf ("%f", &mark2);
```

```
    printf ("Enter mark of subject 3: ");
```

```
    scanf ("%f", &mark3);
```

```
    avg = (mark1 + mark2 + mark3) / 3;
```

```
    printf ("Roll no. : %d \n", roll_no);
```

```
    printf ("Marks: %.2f, %.2f, %.2f \n", mark1, mark2, mark3);
```

```
    printf ("Avg marks : %.2f \n", avg);
```